JAVA INTERVIEW Cheat Sheet– TOPIC WISE

I. Java/JRE/JVM Fundamentals

1. **JVM Basics:** Define the Java Virtual Machine (JVM). Explain the distinctions among JVM, JDK, and JRE.
2. **JVM Architecture:** Detail the JVM architecture, covering its core components (Class Loader, Runtime Data Areas, Execution Engine).
3. **Runtime Data Areas:** List and describe the various JVM runtime data areas (Heap, Stack, Metaspace, Code Cache).
4. **Thread Memory:** Identify the thread-specific runtime data areas and explain their function. Discuss the memory allocation impact of thread creation in these areas.
5. **Java Memory Model (JMM):** Explain the JMM and its memory guarantees. Describe "happens-before" relationships.
6. **Concurrency Implementation:** How does the JVM handle `volatile` variables and atomic operations?
7. **JVM Performance Analysis & Tuning:**
    - **GC Troubleshooting:** How would you diagnose and resolve production service spikes and frequent Full GCs using GC logs?
    - **Tuning Tools:** How do you analyze and tune JVM performance utilizing JFR, JIT logs, and GC logs?
    - **JVM Arguments:** List various Java or JVM arguments useful for performance tuning.
8. **Metaspace:** Define Metaspace and explain how it differs from the deprecated PermGen.

II. Java 8 Features

1. **Functional Interfaces (FI):** Define Functional Interfaces and the rationale for their introduction in Java 8. Is the `@FunctionalInterface` annotation mandatory?
2. **FI Types:** List and categorize different types of FIs. Mention predefined FIs (e.g., `Runnable`). Differentiate between `Predicate`, `Function`, and `Supplier`.
3. **Lambda Implementation:** Write a functional interface and a corresponding lambda expression for a simple task (e.g., summing two numbers or implementing FizzBuzz).
4. **Default Methods:** Explain the purpose and necessity of `default` methods in interfaces.
5. **FI Inheritance:** Can a functional interface inherit from another interface? Explain the rules regarding inheritance and abstract methods for FIs.
6. **Method References:**
    - **Syntax & Use:** Explain method references and their syntax.
    - **Scenario:** Given a list of strings, demonstrate how to sort them case-insensitively using method references instead of a lambda expression.
7. **Lambdas vs. Anonymous Inner Classes:** Compare and contrast lambdas and anonymous inner classes regarding scope and performance.
8. **Asynchronous Programming:** Discuss asynchronous programming in Java, specifically focusing on `CompletableFuture`.
9. **Stream Operations:** Distinguish between terminal and intermediate operations in

Java 8 Streams.

10. **Parallel Streams:** Explain parallel streams. When should they be used? What are the common pitfalls of using `parallelStream()` in a web application context?
11. **Spliterator:** What is a `Spliterator`? Compare and contrast `Iterator` vs. `Spliterator`.
12. **Inheritance in Java 8:** Is multiple inheritance possible in Java 8? If not, why? Discuss multi-level inheritance and the Diamond Problem.

III. Collections Framework

1. **Internal Working:**
    - **`ArrayList`:** Describe the internal mechanics of `ArrayList` (array creation, resizing, and data copying).
    - **`HashMap`:** Explain the internal process of `HashMap` (hashing, index generation, storage in array buckets, hash collisions leading to linked lists/RB trees). Explain the conditions for converting a linked list to an RB tree. Discuss the importance of `hashCode()` and `equals()`.
    - **Key Behavior:** What happens when you use `put()` with a key object that is already present in a `HashMap`?
2. **`ConcurrentHashMap`:** Why use `ConcurrentHashMap` over `Hashtable`? Discuss segmentation, locking, and concurrent modification issues.
3. **List Comparison:** Differentiate between `Vector` and `ArrayList`.
4. **`LinkedList` vs. `ArrayList`:** Provide a scenario where `LinkedList` offers better performance than `ArrayList`.
5. **Map Comparison:** Differentiate between `HashMap` and `Hashtable`.
6. **`TreeMap`:** Explain the internal workings of `TreeMap`. Compare and contrast `TreeMap` and `HashMap`.
7. **Immutable Classes:** What is an immutable class and why are they necessary? How do you secure them against I/O and reflection attacks? Provide examples.
8. **Cloning:** Explain Shallow vs. Deep cloning. When should each be used? Which type is required when creating an immutable class?
9. **Sorting:** Compare `Comparator` vs. `Comparable`. Which one enables *natural* sorting? Implement a `Comparator` for an `Employee` class (with fields like id, name, salary, age) to sort in descending order of salary.
10. **Utility Classes:** Discuss the `Collections` and `Arrays` utility classes. When is it necessary to use synchronized collections?
11. **`CopyOnWriteArrayList`:** What is `CopyOnWriteArrayList` and how does it differ from a standard `ArrayList`?
12. **References:** Explain `WeakHashMap`, `WeakReference`, and `SoftReference`.
13. **Sorting Custom Objects:** Describe how to sort a list of `Customer` objects in an `ArrayList` based on the `customerFirstName` attribute.
14. **Concurrency:**
    - What is the difference between Synchronized Collections and Concurrent Collections?

    - What is the appropriate scenario for using `ConcurrentHashMap`?
15. **Map Operations:** How do you create an empty `Map`?
16. **Collection Removal:** What is the difference between the `remove()` method of `Collection` and the `remove()` method of `Iterator`?
17. **Array vs. ArrayList:** Between an array and `ArrayList`, which is preferred for storing objects?
18. **`Hashtable` Replacement:** Is it possible to replace `Hashtable` entirely with `ConcurrentHashMap`?
19. **`ListIterator`:** Why does `ListIterator` have an `add()` method while `Iterator` does not?
20. **Exceptions:** Why does `ConcurrentModificationException` sometimes occur during iteration?
21. **Conversions:** How do you convert a `Map` to a `List`?
22. **Reverse Map:** How can a `Map` be created with a reverse view and lookup capability?
23. **Shallow Copy:** How do you create a shallow copy of a `Map`?
24. **Generics:** Why is it not possible to create a generic array in Java?
25. **`PriorityQueue`:** What is a `PriorityQueue`?

IV. Exception Handling

1. **Hierarchy:** What is the base class for both `Error` and `Exception` classes in Java?
2. **`finally` Block:** Define the `finally` block. Predict the output of the provided `FinallyTest` class code example.
3. **Types of Exceptions:** Differentiate between Checked and Unchecked Exceptions. Provide examples and explain how each is typically handled.
4. **`finally` Usage:** Can a `finally` block exist without a `catch` block? List scenarios where a `finally` block may *not* execute.
5. **Propagation:** Explain the concept of Exception Propagation with a code example.
6. **Overriding Exceptions:** When overriding a method, is the child class permitted to throw an additional exception not thrown by the parent method?
7. **Try-with-Resources:** Explain the `try-with-resources` construct. Describe any recent changes and when to use it.

V. String Manipulation

1. **Immutability:** Explain the meaning of "immutable" in the context of the Java `String` class. Why is a `String` object immutable?
2. **Object Creation (Basic):** How many objects are created by the following code: `String s=new String("Shiv");` and `String s1="Shiv"; String s2="Shiv";`
3. **Creation Methods:** List the different ways to create a `String` object in Java.
4. **String Interning:** Define String Interning. Why does Java use the String literal concept?
5. **Mutable String Classes:** What is the basic difference between a `String` and a

`StringBuffer` object?

6. **Immutable Class Creation:** Outline the steps to create a custom immutable class in Java.
7. **`toString()`:** What is the use of the `toString()` method?
8. **Efficiency:** Arrange `String`, `StringBuffer`, and `StringBuilder` in order of efficiency for string processing operations.
9. **Interning Output:** Predict the output of the following program regarding equality checks:
   String s1 = "Java";
10. String s2 = "Java";
11. String s3 = new String("Java");
12. String s4 = s3.intern();
13. System.out.println(s1 == s2);
14. System.out.println(s1 == s3);
15. System.out.println(s1 == s4);
16. **Concatenation Output:** Predict the output:
    String s = "Hello";
17. s.concat("World");
18. System.out.println(s);
19. **Object Creation (Advanced):** How many objects are created by:
    String s1 = "abc";
20. String s2 = new String("abc");
21. String s3 = s2.toUpperCase();

VI. Multithreading & Concurrency

1. **Thread Creation:** Compare `Thread` vs. `Runnable`. List different ways to create threads, including using a `Runnable` lambda.
2. **Synchronization Methods:** Describe the various ways to synchronize threads in Java.
3. **`synchronized` Keyword:** What does the `synchronized` keyword do? How does it work internally?
4. **Synchronization Scope:** Differentiate between synchronizing on a method and synchronizing on a block.
5. **Instance vs. Static Synchronization:** Explain the difference between synchronized instance methods and synchronized static methods.
6. **Lock Object:** What object serves as the lock when a method is declared `synchronized`?
7. **Constructors:** Can constructors be synchronized? Justify your answer.
8. **Multiple Synchronized Methods:** What happens if two threads call two different synchronized methods on the same object?
9. **Lock Objects Comparison:** What is the difference between `synchronized(this)` and `synchronized(SomeClass.class)`?
10. **Performance:** Compare the performance impact of `synchronized` versus `ReentrantLock`.
11. **JVM Implementation:** How does the JVM implement synchronization internally

(monitor enter/exit, biased locking)?

12. **`wait()` and Locks:** If a thread holding a lock calls `wait()`, does it release the lock?
13. **Concurrency Issues:** Differentiate between Race Condition and Visibility Problem. How does `volatile` solve the visibility problem?
14. **JMM & Happens-Before:** Discuss the JMM in detail, including "happens-before" relationships.
15. **Daemon Threads:** What is a daemon thread? How can a user thread be converted into a daemon thread after it has started?
16. **Inter-Thread Communication:** Explain `wait()`, `notify()`, and `notifyAll()`. In which class are they present and why?
17. **Tasks:** Compare `Runnable` vs. `Callable`.
18. **`ExecutorService`:** Explain `ExecutorService`. Discuss different types of thread pools and the considerations for creating one.
19. **Pausing Threads:** What is the fundamental difference between `wait()` and `sleep()` methods?
20. **Locks vs. `synchronized`:** Compare the `Lock` interface with the intrinsic lock (`synchronized`).
21. **`ReentrantLock`:** Explain the difference between `ReentrantLock` and intrinsic locks (covering explicit lock/unlock, fairness policy, and condition variables).
22. **Reentrant Locks:** Define a reentrant lock. Why is it called reentrant? What is its main purpose? Discuss fairness and the difference with `ReentrantReadWriteLock`.
23. **`ThreadLocal`:** What is the use of the `ThreadLocal` class?
24. **Sequencing Threads:** How do you ensure that two threads, T1 and T2, run in sequence (T1 then T2) in Java?
25. **Synchronization Utilities:** What is the difference between `CyclicBarrier` and `CountDownLatch`?

VII. Miscellaneous Java

1. **Serialization:** Explain `Serializable`. Discuss the significance of `serialVersionUID`. What is `Externalizable`?
2. **Marker Interfaces:** What are marker interfaces and why are they needed?
3. **Operators:** Is there a difference between `a = a + b` and `a += b` expressions?
4. **Overriding Exceptions (Runtime):** If a superclass method throws a `NullPointerException` (an unchecked exception), can the overriding method in the subclass throw a `RuntimeException`?
5. **`volatile` Arrays:** Can an array be declared `volatile`? What are the caveats, and how can you properly mark an array volatile in Java?
6. **Cloning (Advanced):** Which class contains the `clone()` method (`Cloneable` or `Object`)? Is `Cloneable` considered a "broken interface"?
7. **Floating-Point Math:** What will `5 * 0.1 == 0.5` return (true or false)?

VIII. Spring/Spring Boot Core

1. **IoC/DI:** Explain Inversion of Control (IoC). What are the purpose and benefits of the IoC container? Define Dependency Injection (DI) and list its types.
2. **Singleton:** Compare the Singleton pattern in Spring with a standard Java Singleton implementation. Is it safe to assume a Singleton bean is thread-safe in Spring Framework?
3. **Bean Lifecycle:** Describe the complete lifecycle of a Spring bean.
4. **Autowiring:**
    - **Mechanisms:** Explain Autowiring and its different methods.
    - **Conflict Resolution:** If two classes implement the same interface and you need to inject one of them using autowiring, how do you resolve the conflict? Discuss `@Primary`, `@Qualifier`, and convention over configuration (qualifying by object name).
5. **Configuration:** Explain the role of `@Configuration` and `@Bean` annotations.
6. **Injection Annotations:** Compare `@Autowired` vs. `@Inject`.
7. **HTTP Response Body:** Differentiate between `@ResponseBody` and `ResponseEntity`.
8. **Controllers:** Compare `@Controller` and `@RestController`. Why is `@ResponseBody` not explicitly needed in `@RestController`?
9. **`@SpringBootApplication`:** Explain the significance of the `@SpringBootApplication` annotation. List the annotations it comprises and the function of each.
10. **`@ComponentScan`:** How does `@ComponentScan` work if no package name is specified?
11. **`@EnableAutoConfiguration`:** How does `@EnableAutoConfiguration` facilitate plug-and-play functionality in Spring Boot?

IX. Spring Controller & Web Layer (REST APIs)

1. **Response Handling:** How are JSON and XML responses handled in Spring Boot REST APIs?
2. **Parameter Injection:** Differentiate between `@RequestParam`, `@PathVariable`, and `@RequestBody`. Provide an implementation example for each, including endpoint generation and value passing.
3. **Best Practices:** Outline best practices for creating endpoints and URL naming schemes.
4. **REST Methods:** Describe the various REST methods (GET, PUT, POST, PATCH, OPTIONS, DELETE). Which of these are idempotent?
5. **File Operations:** How do you implement file upload and download functionality in Spring REST?
6. **CORS:** How is Cross-Origin Resource Sharing (CORS) configured in Spring Boot REST APIs?
7. **Serialization:** How does Spring Boot automatically configure Jackson for JSON serialization/deserialization?
8. **Request Lifecycle:** Explain the lifecycle of a REST request in Spring Boot (DispatcherServlet -> HandlerMapping -> Controller -> ViewResolver).
9. **HATEOAS:** Explain the Richardson Maturity Model and HATEOAS. How do you

implement HATEOAS in Spring Boot REST?

10. **Security:** How are REST APIs secured using Spring Security and JWT? Describe the JWT structure and lifecycle.
11. **Rate Limiting/Throttling:** How do you implement rate limiting and throttling in a Spring Boot application? What steps are required?
12. **Caching:**
    - **Basics:** How do you handle caching with Spring Boot? Explain `@EnableCaching`.
    - **Annotations:** How does `@Cacheable` work? Differentiate between `@Cacheable`, `@CachePut`, and `@CacheEvict`. Discuss cache expiration handling.
    - **Conditional Caching:** How do you implement conditional caching using the `unless` and `condition` attributes of `@Cacheable`?
    - **Clustered Caching:** How do you handle caching in a clustered environment? How do you prevent cache stampede or cache avalanche?
    - **ETags:** How do you implement ETags and conditional GET requests with caching?
    - **Internal Working:** How does `@Cacheable` work internally (CacheInterceptor, Cache Manager, Cache Resolver, lookup decision)?
    - **Multi-Level Caching:** How do you implement multi-level caching (e.g., in-memory + Redis)? When would you use it, and what are the disadvantages?
13. **`DispatcherServlet`:**
    - **Role & Lifecycle:** Explain the role of the `DispatcherServlet` and its lifecycle from initialization to request handling.
    - **Asynchronous:** How does `DispatcherServlet` handle asynchronous requests (`DeferredResult`, `Callable`)?
    - **Interceptors:** How does `DispatcherServlet` interact with `HandlerInterceptor`, and what is the exact order of execution?
14. **Validation:**
    - **Controller Validation:** How do you validate request data in a Spring Boot controller? Compare `@Valid` and `@Validated`.
    - **Custom Validation:** How do you implement custom validation annotations?
    - **Nested Validation:** How do you validate nested objects and collections (e.g., list of objects) in a request payload?
    - **Parameter Validation:** How do you validate path variables and query parameters?
    - **Cross-Field Validation:** How do you implement validation across multiple fields (e.g., password and confirm password)?
    - **Advanced Validation:** How do you customize the `MethodValidationPostProcessor`?
    - **File Validation:** How do you validate file uploads or multipart requests in Spring?

X. Exception Handling (Spring Boot)

1. **Local Handling:** How do you handle exceptions in Spring Boot using `@ExceptionHandler`?
2. **Global Handling:** Differentiate between `@ControllerAdvice` and `@RestControllerAdvice`.
3. **Custom Exceptions:** How do you create and handle custom exceptions?
4. **Validation Exceptions:** How are validation exceptions (`MethodArgumentNotValidException`) handled?
5. **Propagation:** How do you propagate exceptions from the service layer to the controller layer?
6. **Centralized Handling:** How do you use `ResponseEntityExceptionHandler` for centralized exception handling?

XI. JPA/JDBC & ORM

1. **JPA Ecosystem:** What is the difference between JPA, Hibernate/MyBatis, and Spring Data JPA?
2. **`transient` Keyword:** What is the use of the `transient` keyword?
3. **Entity Creation:** Create an `Employee` Entity (id, name, age, salary) with all necessary JPA annotations.
4. **ID Generation:**
    - **Purpose & Strategies:** What is the purpose of `@GeneratedValue`? List and explain the differences between `GenerationType.AUTO`, `IDENTITY`, `SEQUENCE`, and `TABLE`.
    - **AUTO Logic:** How does `GenerationType.AUTO` determine which strategy to use?
    - **SEQUENCE vs. IDENTITY:** When would you choose `GenerationType.SEQUENCE` over `IDENTITY`, and what are the performance implications?
    - **Generators:** What is the role of `@SequenceGenerator` and `@TableGenerator`?
    - **Custom Generator:** How do you implement a custom ID generator in JPA?
5. **Repositories:** How does `JpaRepository` differ from `CrudRepository`?
6. **Custom Queries (JPQL):** How do you use the `@Query` annotation for custom JPQL queries?
7. **Pagination/Sorting:** How do you enable pagination and sorting in Spring Data JPA?
8. **`EntityManager`:** What is the role of `EntityManager` in Spring Data JPA?
9. **Relationships:** When should you use `@OneToOne`, `@OneToMany`, `@ManyToOne`, and `@ManytoMany`?
10. **Fetching:** Explain the difference between `fetch = FetchType.LAZY` and `FetchType.EAGER`.
11. **N+1 Problem:** Explain the N+1 query problem. Is it caused by `FetchType.EAGER` or lazy loading?
12. **Modifying Queries:** If you use `@Query` to modify data, what other annotations are required? Discuss the use of `@Modifying` and `@Transactional`.

13. **Auditing:** How do you implement auditing in Spring Data JPA (using `@CreatedDate`, `@LastModifiedDate`)?
14. **Locking:**
    - **Types:** How do you handle optimistic and pessimistic locking?
    - **Annotations:** Discuss `@Version`, `@Lock`, `@Transactional`, and various lock modes.
15. **Data Sources:** How do you integrate Spring Data JPA with multiple data sources?
16. **Multi-Tenancy:** How do you implement multi-tenancy? Discuss strategies like DB per tenant, Schema per tenant, and shared schema with a discriminator.
17. **Native Queries/Projections:** How does Spring Data JPA work with native queries and projections?
18. **Optimization:** How do you debug and optimize slow queries?
19. **Bidirectional Relationships:** How do you handle bidirectional relationships and avoid infinite recursion in JSON serialization?
20. **Cascade Operations:** How do you implement cascade operations in JPA?
21. **Entity Graphs:** How do you use Entity Graphs for optimizing queries?
22. **Batch Operations:** How do you implement efficient batch inserts/updates?
23. **Named Queries:**
    - **Definition:** How do you define a Named Query using `@NamedQuery`?
    - **Types:** What is the difference between `@NamedQuery` and `@NamedNativeQuery`?
    - **Resolution & Precedence:** How does Spring Data JPA resolve Named Queries internally? What is the precedence between Named Queries and the `@Query` annotation, and how do you override a Named Query?
24. **Indexing (JPA):**
    - **Definition:** How do you define an index using annotations?
    - **Column vs. Index:** What is the difference between `@Index` and `unique=true` on `@Column`?
    - **Impact:** Discuss the advantages and disadvantages of indexing. What is the impact of indexing on insert/update performance?
    - **Foreign Keys:** Does JPA automatically create indexes for foreign keys?

XII. Database Concepts (SQL)

1. **DML/DDL:** Differentiate between DDL and DML.
2. **Removal:** What is the difference between `DELETE` and `TRUNCATE`?
3. **Filtering:** Differentiate between `WHERE` and `HAVING` clauses.
4. **Joins:** Explain `INNER JOIN` vs. `LEFT JOIN` with examples.
5. **Properties:** What are the ACID properties?
6. **Indexing:** When should indexing be used? List advantages and disadvantages.
7. **Pagination:** How do you implement pagination in SQL?
8. **Views:** Explain Views over tables. Why are they used? Compare Views vs. Materialized Views (Ad/Disad).
9. **Execution Order:** Arrange the following SQL clauses in the correct order of execution: `WHERE, GROUP BY, SELECT, HAVING, FROM, JOIN, ORDER BY, LIMIT/OFFSET`.

10. **Self Join:** Explain Self Join and its necessity.
11. **Ranking Queries:**
    - Find the name of the employee with the second (or Nth) maximum salary. (Schema: `Employee[id, name, salary, dept]`)
    - Find employees who earn more than the average salary of their department, and rank them within their department by salary.
    - Find the top 3 highest-paid employees in each department.
12. **Advanced Filtering Queries (Schema: `Employee(id, name, department_id, manager_id, salary, hire_date)`):**
    - Find employees who earn more than the average salary of their department.
    - Find employees who joined before the earliest hire date in their department.
    - Find employees who earn more than the average salary of their department AND more than their manager.
    - Find employees who have the same salary as someone in another department.
13. **Partitioning:**
    - **Types:** Explain partitioning and its different types (Range, List, Hash).
    - **Orientation:** What is the difference between horizontal partitioning and vertical partitioning?
    - **Indexes:** How do you manage indexes on partitioned tables?
    - **Sub-partitioning:** What is sub-partitioning and when is it used?
    - **Composite Keys:** How do you implement partitioning with composite keys?
14. **Keys:** Differentiate between Composite Keys, Primary Keys, Candidate Keys, and Surrogate Keys. Is it possible to have a table without a primary key? If so, what are the implications, and how would you uniquely identify rows?

XIII. Microservices Architecture

1. **Definition & Comparison:** Define microservices in your own words. Contrast it with the Monolithic architecture. List the benefits and downsides of using microservices.
2. **Communication:** How do microservices communicate? Discuss synchronous (REST/HTTPS, gRPC) and asynchronous (Message queues like RabbitMQ, Kafka) modes.
3. **Event-Driven Architecture (EDA):**
    - **Explanation:** Explain EDA (How, Why, and When is it used?).
    - **Concepts:** Explain the difference between an event, a command, and a query in EDA.
    - **Broker:** What is the role of an Event Bus or Message Broker?
    - **Event Sourcing:** Explain Event Sourcing and how it differs from traditional CRUD.
    - **Distributed Transactions:** What are the challenges of implementing distributed transactions in EDA, and how is the Saga pattern used to solve them?
4. **Saga Pattern:**
    - **Details:** Explain the Saga pattern (types, use cases).
    - **Failure Handling:** What happens if one step in a Saga fails? How does the Saga ensure eventual consistency?
    - **Compensation:** How do you implement compensating transactions? How do

you handle retries and idempotency in Saga steps?
    - **Challenges:** What are the challenges of implementing Saga in a high-throughput system? How would you design a Saga for a globally distributed system with strict latency requirements?
5. **CQRS (Command Query Responsibility Segregation):**
    - **Details:** Explain CQRS. Differentiate between Command and Query. Discuss benefits and disadvantages.
    - **Consistency:** How do you keep the read model synchronized with the write model? Explain how eventual consistency works in CQRS.
6. **Feign Client:** How is a Feign Client implemented? What are the steps? Where are service URLs managed? How do you implement fallbacks?
7. **Resilience:**
    - **Definitions:** Differentiate between fault tolerance and resilience.
    - **Patterns:** How do you ensure resilience in microservices? List various resilience patterns (Circuit Breaker, Bulkhead).
8. **Load Balancing:**
    - **Types:** Explain client-side and server-side load balancing.
    - **Example:** Describe a load balancing scheme used in one of your projects.
    - **Comparison:** How does Kubernetes Service load balancing differ from an API Gateway load balancer?
    - **Layers:** What is the difference between Layer 4 and Layer 7 load balancing? Which is better for microservices?
9. **Circuit Breaker:** Detail the steps to implement a Circuit Breaker library (e.g., Resilience4j or Hystrix). Discuss the handling of various states.
10. **Mitigation Techniques:** Explain how retry, retry with exponential backoffs, timeouts, and rate limiting help resilience. How are they implemented?
11. **Scaling:**
    - **Types:** Discuss Horizontal vs. Vertical scaling.
    - **MS Scalability:** Are microservices inherently more scalable than a monolith? Why?
    - **Auto-Scaling:** How do you implement auto-scaling in a Kubernetes-based microservices architecture (HPA, VPA, CA)?
12. **Design Principles:** Explain the 12 Factor App standards and the CAP theorem.
13. **Service Discovery:** What is service discovery, and why is it needed? Outline the steps to implement it in Spring Boot.
14. **Distributed Tracing:**
    - **Importance:** What is distributed tracing, and why is it crucial in microservices?
    - **Concepts:** Explain the concepts of TraceID and SpanID. How do you propagate the trace context through all microservices?
    - **Implementation:** How is distributed tracing implemented in Spring Boot? Which libraries are used?